Computer Network Course Practice

Course Overview & Network Programming Project

Instructor: Zhengyang Liu

School of Computer Science and Technology
Beijing Institute of Technology

2026 Fall

Course Information

- **Title:** Computer Network Course Practice
- **Code:** 100073009
- **Credits / Hours:** 1 credit / 32 hours
- **Type:** Compulsory
- **Pre-requisites:**
  - Computer Networks
  - Programming in C++ / Java / Python
  - Basic Socket and Multi-Process / Multi-Thread concepts

Course Website

- Course platform: `https://lexue.bit.edu.cn/`
- Course page: `https://lexue.bit.edu.cn/course/view.php?id=17077`
- **Login name:** your student ID
- **Login password:** your own password
- **Course pass:** See the current course announcement

Use the course website for

Downloading PPT/PDF materials, checking announcements, and uploading homework, project files, and reports.

Course Description

- A practical course in computer networks.
- Network programming
  - implementation of protocols and services;
  - development and testing of a complete network application.

Goals of This Course

- Understand computer network architectures and protocols more deeply through practice.
- Build the basic ability of network engineering, programming, and application.
- Be able to design a network and correctly configure switches and routers.
- Be able to develop, test, and explain a basic network application.

Understand the protocol, implement it correctly, and verify it experimentally.

Project: Network Programming

Design and Implementation of a Lightweight Email Client

SMTP + POP3 + Socket Programming

Complete the core functions first.
Advanced functions are bonus features.

Why This Project?

- A practical application of TCP socket programming.
- Learn how real application-layer protocols exchange commands and responses.
- SMTP and POP3 are simple, text-based, and still represent real Internet protocols.
- Build a complete client without reproducing a commercial mail system.

Required scope: one account, plain-text email, SMTP sending, POP3 receiving, a simple Inbox, and local storage.
Use a local or instructor-provided test server whenever possible; TLS/OAuth is not part of the basic requirement.

Project Structure

Basic Requirements

- One account / one test server
- Send email with SMTP
- Receive email with POP3
- Simple Inbox and message viewer
- Save messages locally
- Error handling and logging

Bonus Features

- TLS / SSL
- Attachments and MIME
- Background checking
- IMAP
- Multiple accounts
- Search, GUI, HTML email, etc.

Basic Requirement 1: Send Email

- Connect to an SMTP server using a TCP socket.
- Let the user enter recipient, subject, and message body.
- Implement a small subset of SMTP commands:
  - `HELO/EHLO`
  - `MAIL FROM`
  - `RCPT TO`
  - `DATA`
  - `QUIT`
- Parse and check server reply codes.

Do not use a high-level library that completely hides SMTP in the required implementation.

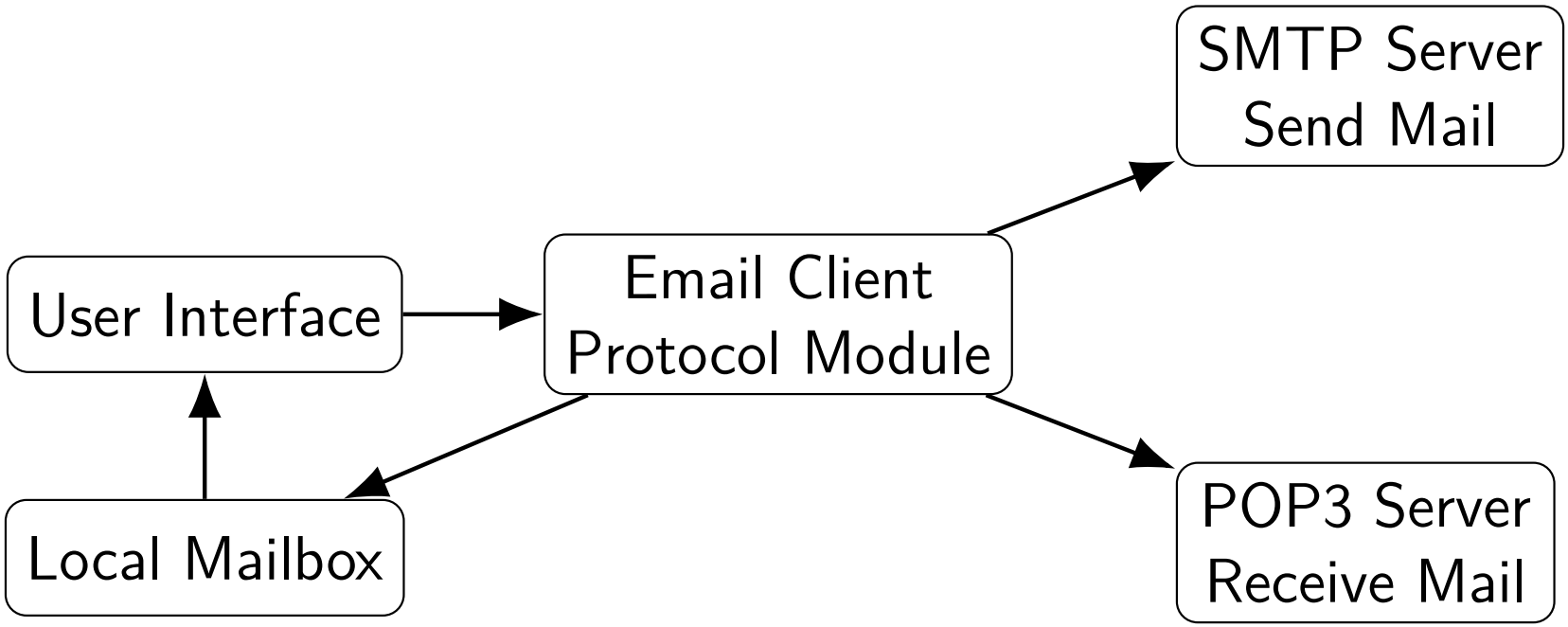
Basic Requirement 2: Receive Email

- Connect to a POP3 server and authenticate.
- Implement a small subset of POP3 commands:
  - `USER / PASS`
  - `STAT` or `LIST`
  - `RETR`
  - `QUIT`
- Display a simple Inbox:
  - message number, sender, subject, and date when available;
  - let the user select and read one message.

Basic Requirement 3: Local Mailbox

- Save downloaded messages locally.
- Reload saved messages after restarting the program.
- Storage can be simple: text files, JSON, or SQLite.
- A command-line interface is completely acceptable.
- Handle connection failures, wrong passwords, timeouts, and invalid responses gracefully.
- Keep a short log; **never log passwords**.

Protocol Workflow



Focus on command–response behavior and correct TCP stream handling.

Remember: TCP is a byte stream; one send does not necessarily equal one recv.

Bonus Features

- TLS / STARTTLS
- MIME attachments
- Background mail checking
- IMAP support
- Multiple accounts
- Local search
- Read/unread management
- HTML email
- Graphical interface
- Improved multithreading

Principle

A smaller number of well-implemented bonus features is better than many incomplete features.

Testing and Deliverables

- Demonstrate successful connection to both SMTP and POP3 servers.
- Demonstrate at least three successful sent messages.
- Retrieve and display received messages.
- Save messages locally and reopen them after restart.
- Demonstrate at least three error cases.
- Briefly measure normal send/retrieve time; a small table is enough and stress testing is not required.
- Submit individually:
  - source code and README;
  - one solution report;
  - screenshots or terminal output;
  - test results and representative logs;
  - description of bonus features, if any.

Grading Policy

Final Grade (100 Points)	
Report	**40%**
Results (Project Code)	**20%**
Presentation (Compulsory)	**10%**
Course Performance	**30%**

The report is a major part of the final grade—do not treat it as an afterthought.

Report Structure

Use the course report template. Keep the following section names and adapt the content to the email-client project.

1. Requirements
2. Analysis
3. Design
4. Development and Implementation
5. System Deployment, Startup, and Use
6. System Test
7. Performance
8. Summary or Conclusions
9. References
10. Comments (optional)

References: Cite the protocol documents, libraries, and other materials actually used. No fixed minimum count is required unless announced separately.

Presentation

- Compulsory.
- Explain the architecture and SMTP/POP3 protocol behavior.
- Demonstrate the working system, including normal and error cases.
- Be ready to explain your own code and TCP stream handling.

Assignments

- All assignments are **individual**.
- Start early.
- Ask questions early.
- Submit on time.

Why start early?

Network programs often fail because of small protocol, timeout, parsing, or environment issues. These problems are easier to solve when you leave time to test them carefully.

IMPORTANT!

- Prepare for the experiment before starting it.
- C/C++ and Java are encouraged; other languages are allowed.
- A beautiful UI is a plus, but networking correctness comes first.
- A detailed report is very important.
- Copying from others may receive zero credit.
- Do not download and submit existing project code from the Internet.
- Be prepared to explain your implementation during the demo or presentation.

What to Remember

1. **Understand the protocol.**
2. **Make the basic system work correctly.**
3. **Test normal and error cases carefully.**
4. **Explain your design and implementation clearly.**

Correctness first.

Features second.